

This is a copy of a conversation between ChatGPT & Anonymous.

Report conversation

I don't understand why this cast is required:

```
def backfill(
    self: Self, tolerance: float | Iterable[float]
) -> T_Xarray:
    """Backward fill new values at up-sampled indices.

    Parameters
    -----
    tolerance : float | Iterable[float]
        Maximum distance between original indices and
        the up-sampling method.
        Up-sampled data with indices that are within
        ``abs(index[indexer] - target) <= tolerance`` are
        new values. Data with indices that are greater than
        tolerance are filled with ``NaN``.

    Returns
    -----
    backfilled : DataArray or Dataset
    """
    obj = self._drop_coords()
    (grouper,) = self.groupers
    return cast(
        T_Xarray,
        obj.reindex(
            {self._dim: grouper.full_indexer},
        ),
    )
```

Without it, we get this error:

```
xarray/core/resample.py: note: In member "b:
xarray/core/resample.py:118: error: Incompa
```

For context, `obj` is a `T_Xarray`, and `T_Xarray` is:

```
T_Xarray = TypeVar("T_Xarray", bound=Union[
```

Each of `DataArray` & `Dataset` have their own `.reindex` method, which each return `T_DataArray` & `T_Dataset` respectively.

Those are defined as:

```
T_Dataset = TypeVar("T_Dataset", bound="Data
T_DataArray = TypeVar("T_DataArray", bound=
```

So I can't see why it doesn't see the result as matching `T_Xarray`.

The issue you're encountering is related to the way that Python's type checking and generics work. The error message you're seeing is due to the type checker not being able to guarantee that the type returned by `obj.reindex` is the same as `T_Xarray`.

Here's a breakdown of the situation:

1. `T_Xarray` is a generic type that can be either `DataArray` or `Dataset`.
2. Even though both `DataArray` and `Dataset` have a `.reindex` method that returns their respective types, the type checker cannot guarantee that the return type of `obj.reindex` is the exact same type as `obj` (i.e., `T_Xarray`).

To illustrate this with a simplified example:

Suppose we have two classes, `A` and `B`, and a generic type `T` that can be either `A` or `B`:

python

```
T = TypeVar("T", bound=Union[A, B])
```

 Copy code

Both `A` and `B` have a method `foo()` that returns their respective types:

python

```
class A:
    def foo(self) -> A:
        ...

class B:
    def foo(self) -> B:
        ...
```

 Copy code

Now, if we have a function that takes an argument of type `T` and calls `foo()` on it:

python

```
def bar(obj: T) -> T:
    return obj.foo()
```

 Copy code

The type checker has no guarantee that the returned type of `obj.foo()` is the same as `T`. This is because, from the type checker's perspective, `T` could be either `A` or `B`, and it cannot ensure that the type of `obj.foo()` matches the exact type of `obj`.

This is why the `cast` is required in your code. By using `cast(T_Xarray, ...)`, you're telling the type checker: "Trust me, the returned type here is the same as `T_Xarray`".

In conclusion, even though it might seem obvious to us that the return type matches, the type checker cannot make that assumption based on the information provided. The `cast` is a way of providing an explicit guarantee to the type checker.

☰ ChatGPT 5.2 ▾

OK, thanks for the explanation.

But my confusion is that `obj.foo()` can't change the type, because each of `A` & `B` return the same type...

You're correct in your understanding, and your intuition is on point. If `A`'s method `foo()` always returns an instance of `A` and `B`'s method `foo()` always returns an instance of `B`, then calling `foo()` on an instance of `T` (which is either `A` or `B`) should always return an instance of `T`.

However, the type checker (like `mypy`) doesn't make this inference automatically. Here's why:

When we declare a type variable `T` that can be bound to either `A` or `B`, the type checker sees methods called on `T` as potentially being from either `A` or `B`. Even if both classes have a method of the same name, the type checker doesn't make the assumption that they have the same behavior or return type.

So, in the function:

python

 Copy code

```
def bar(obj: T) -> T:  
    return obj.foo()
```

When checking this function, the type checker sees the call `obj.foo()` and recognizes that `foo()` could be the method from either `A` or `B`. Since it doesn't make assumptions about the implementation of these methods, it can't guarantee that the return type of `obj.foo()` is `T`.

This is a limitation of the type checker's inference, and it's why the `cast` is necessary. The `cast` serves as a manual override, telling the type checker "I know what I'm doing, trust me that the type here is correct."

In an ideal world, the type checker would be smart enough to recognize the pattern you've described and not require the `cast`. However, type checkers like `mypy` are designed to be conservative to avoid false negatives, so they err on the side of caution.

It's worth noting that using `cast` isn't a "fix" in the traditional sense—instead, it's a way to bypass a limitation of the type checker. The real guarantee of type correctness still lies with the developer's understanding of the code.